



(12) 发明专利申请

(10) 申请公布号 CN 117891588 A

(43) 申请公布日 2024. 04. 16

(21) 申请号 202311413597.8

(22) 申请日 2023.10.30

(71) 申请人 山东浪潮数据库技术有限公司

地址 250100 山东省济南市高新区浪潮路
1036号S02楼5楼办公区

(72) 发明人 苑晓龙 金宁

(74) 专利代理机构 济南信达专利事务所有限公
司 37100

专利代理师 阚恭勇

(51) Int.Cl.

G06F 9/50 (2006.01)

G06F 9/54 (2006.01)

H04L 67/1008 (2022.01)

H04W 28/084 (2023.01)

G06F 16/2458 (2019.01)

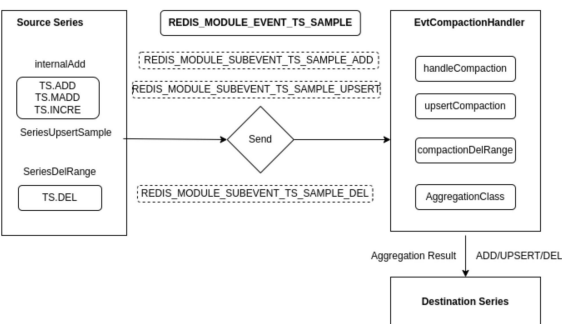
权利要求书2页 说明书4页 附图4页

(54) 发明名称

一种基于OpenCL实现时序数据降采样预计算
方法

(57) 摘要

本发明提供一种基于OpenCL实现时序数据降采样预计算方法,属于时序数据库技术领域,RedisTimeseries通过为时间序列数据设置相应的合并规则(CompactionRule)实现降采样预计算的特性;支持通过“TS.CREATERULE”指令指定源键、目标键、降采样算子、固定时间窗口等创建合并规则,也支持在默认配置文件中“redis.conf”设置“COMPACTION_POLICY”为源键指向的时间序列数据指定多个合并规则。



1. 一种基于OpenCL实现时序数据降采样预计算方法,其特征在于,
RedisTimeseries通过为时间序列数据设置相应的合并规则(CompactionRule)实现降采样预计算的特性;

通过“TS.CREATE RULE”指令指定源键、目标键、降采样算子、固定时间窗口创建合并规则,或在默认配置文件中“redis.conf”设置“COMPACTION_POLICY”为源键指向的时间序列数据指定数个合并规则。

2. 根据权利要求1所述的方法,其特征在于,
当通过“TS.ADD/TS.MADD/TS.INCRE”指令新增或更新数据点(指定时间戳的指标数据)时,合并规则会根据指定的降采样算子进行相应的降采样聚合计算。

3. 根据权利要求1或2所述的方法,其特征在于,
合并规则会随着时间序列数据序列化到磁盘文件中,也可以通过读取磁盘文件来加载。

4. 根据权利要求3所述的方法,其特征在于,
通过Redis Module的API增加了RedisModule_FireServerEvent函数来发送事件,定义了REDISMODULE_EVENT_TS_SAMPLE类型事件,包括如下三种事件

REDISMODULE_SUBEVENT_TS_SAMPLE_ADD

REDISMODULE_SUBEVENT_TS_SAMPLE_UPSERT

REDISMODULE_SUBEVENT_TS_SAMPLE_DEL

“TS.ADD/TS.MADD/TS.INCRE”、“TS.DEL”指令新增或删除时序数据点(SAMPLE),同步调用SeriesAddSample、SeriesUpsertSample、SeriesDelRange函数分别会触发ADD、UPSERET、DEL三种事件。

5. 根据权利要求4所述的方法,其特征在于,
在RedisModule_Onload函数中通过RedisModule_SubscribeToServer函数以回调函数的方式来注册上述事件的处理器EvtCompactionHandler,它根据上述不同的事件类型分别调用handleCompaction、upsertCompaction、compactionDelRange函数,下一步依据OpenCL启用标识来决定是否协调GPU异步执行相应的降采样聚合算法内核程序。

6. 根据权利要求1所述的方法,其特征在于,
定义CPU、GPU异构处理器共享内存区域的内存管理器CLMemoryManager用来为CLAggSum、CLAggAvg、CLAggFirst、CLAggLast的降采样聚合算法内核的cl_context结构体申请和回收内存。

7. 根据权利要求6所述的方法,其特征在于,
内存管理器也管理CLSeries结构体中的Chunk结构体的内存空间,该空间为CPU与GPU的共享内存空间,避免在异构设备CPU及GPU之间发生数据拷贝。

8. 根据权利要求7所述的方法,其特征在于,
CompactionRule结构体由CPU来管理,CPU调用OpenCL标准的CLEnqueueMapBuffer函数将上述申请的共享内存空间映射为CompactionRule结构体中间接引用中的cl_context结构体的指针,以便GPU访问该共享内存空间。

9. 根据权利要求8所述的方法,其特征在于,
CompactionRule结构体中定义了cl_compaction标识定义该时序数据序列是否采用

OpenCL标准AggregationClass结构体实现的CLAggSum、CLAggAvg、CLAggFirst降采样聚合算法内核；若需要降采样预计算的时间序列的量较多，GPU的算力无法满足时，通过动态调整cl_compaction标识降低GPU的负载，达到快速执行内核程序响应CPU协同的目的。

10. 根据权利要求9所述的方法，其特征在于，

CPU框内的函数由CPU负责执行，GPU框内的函数是基于OpenCL标准的内核函数；CPU与GPU除前述采用共享内存机制管理降采样算法上下文及时序数据块(chunk)外，GPU采用cl_command_queue内核执行队列异步接受协调指令调用执行内核函数。

一种基于OpenCL实现时序数据降采样预计算方法

技术领域

[0001] 本发明涉及时序数据库技术领域,尤其涉及一种基于OpenCL实现时序数据降采样预计算方法。

背景技术

[0002] 大多数物联网应用场景要求时序数据的实时性,实时性越高,数据产生的价值越大。时序数据采集实时性与数据量之间呈现正相关的关系。时序数据库中针对海量时序数据实时降采样预计算会对边缘计算设备的CPU造成较高的负载压力。边缘计算设备封装的处理器芯片一般都包含GPU(如ARM Mali系列);RedisTimeSeries是在Redis内存数据库的基础上扩展的支持时序数据结构的模块,它提供了支持降采样预计算规则的特性来支撑降采样预计算,其采用CPU单线程实现时序数据降采样预计算,造成CPU资源较高的负载压力,影响了时序数据的入库效率,然而GPU此时处于空闲状态。

[0003] 随着物联网技术的发展,越来越多的物(设备或传感器等)相互连接起来,从而产生了大量的时序数据,时间序列数据是指是一段时间内某个特定指标量的一系列数值数据点,通过指标和标签以及时间戳来唯一标识一个指标量,这些指标数据具有不可变性。OpenCL(Open Computing Language)是一个开放、免费的标准,它用于大型机、云服务器、移动和嵌入式设备中各种加速器的异构并行计算领域。RedisTimeseries内存时序数据库采用CPU单线程实现时序数据的存储和降采样预计算,降采样预计算占用了较多的CPU资源,影响了时序数据的入库效率而此时GPU处于空闲状态。

发明内容

[0004] 为提升海量时序数据入库效率和加速降采样预计算的需求,本发明一种基于OpenCL实现的时序数据降采样预计算方法,目的是设计实现一种利用实现了OpenCL技术标准的GPU设备的并行计算能力,利用空闲GPU资源基于OpenCL技术将RedisTimeSeries数据库来加速降采样预计算,降低数据库进程的CPU的负载,提高时序数据入库的实时性。

[0005] 本发明的技术方案是:

[0006] 一种基于OpenCL实现时序数据降采样预计算方法,RedisTimeseries通过为时间序列数据设置相应的合并规则(CompactionRule)实现降采样预计算的特性。通过“TS.CREATERULE”指令指定源键、目标键、降采样算子、固定时间窗口创建合并规则,或在默认配置文件中“redis.conf”设置“COMPACTION_POLICY”为源键指向的时间序列数据指定数个合并规则。

[0007] 进一步的,

[0008] 当通过“TS.ADD/TS.MADD/TS.INCRE”指令新增或更新数据点(指定时间戳的指标数据)时,合并规则会根据指定的降采样算子进行相应的降采样聚合计算。

[0009] 合并规则也会随着时间序列数据序列化到磁盘文件中,同样也可以通过读取磁盘文件来加载。

- [0010] 再进一步的,
- [0011] 通过Redis Module的API增加了RedisModule_FireServerEvent函数来发送事件,定义了REDISMODULE_EVENT_TS_SAMPLE类型事件,包括如下三种事件
- [0012] REDISMODULE_SUBEVENT_TS_SAMPLE_ADD
- [0013] REDISMODULE_SUBEVENT_TS_SAMPLE_UPSERT
- [0014] REDISMODULE_SUBEVENT_TS_SAMPLE_DEL
- [0015] “TS.ADD/TS.MADD/TS.INCRE”、“TS.DEL”指令新增或删除时序数据点(SAMPLE),同步调用SeriesAddSample、SeriesUpsertSample、SeriesDelRange函数分别会触发ADD、UPSERET、DEL三种事件。
- [0016] 在RedisModule_Onload函数中通过RedisModule_SubscribeToServer函数以回调函数的方式来注册上述事件的处理器EvtCompactionHandler,它根据上述不同的事件类型分别调用handleCompaction、upsertCompaction、compactionDelRange函数,下一步依据OpenCL启用标识来决定是否协调GPU异步执行相应的降采样聚合算法内核程序。
- [0017] 定义CPU、GPU异构处理器共享内存区域的内存管理器CLMemoryManager用来为CLAggSum、CLAggAvg、CLAggFirst、CLAggLast的降采样聚合算法内核的cl_context结构体申请和回收内存。
- [0018] 内存管理器也管理CLSeries结构体中的Chunk结构体的内存空间,该空间为CPU与GPU的共享内存空间,避免在异构设备CPU及GPU之间发生数据拷贝。
- [0019] CompactionRule结构体由CPU来管理,CPU调用OpenCL标准的CLEnqueueMapBuffer函数将上述申请的共享内存空间映射为CompactionRule结构体中间接引用中的cl_context结构体的指针,以便GPU访问该共享内存空间。
- [0020] CompactionRule结构体中定义了cl_compaction标识定义该时序数据序列是否采用OpenCL标准AggregationClass结构体实现的CLAggSum、CLAggAvg、CLAggFirst降采样聚合算法内核;若需要降采样预计算的时间序列的量较多,GPU的算力无法满足时,通过动态调整cl_compaction标识降低GPU的负载,达到快速执行内核程序响应CPU协同的目的。
- [0021] CPU框内的函数由CPU负责执行,GPU框内的函数是基于OpenCL标准的内核函数;CPU与GPU除前述采用共享内存机制管理降采样算法上下文及时序数据块(chunk)外,GPU采用cl_command_queue内核执行队列异步接受协调指令调用执行内核函数。
- [0022] 本发明的有益效果是
- [0023] 充分利用边缘端空闲GPU处理器实现的降采样聚合算法内核,与CPU处理器通过共享内存存储空间的方式协同处理,加速降采样预计算的同时释放了CPU的算力,提升了海量时序数据的入库效率。在物联网应用场景中,需要海量的边缘端设备存储时序数据,该方法可降低边缘设备的硬件成本。

附图说明

- [0024] 图1是REDISMODULE_EVENT_TS_SAMPLE事件驱动示意图;
- [0025] 图2是CPU/GPU共享全局内存协同处理示意图;
- [0026] 图3是合并规则和降采样算法结构体类图;
- [0027] 图4是ADD事件处理器CPU/GPU异步协同工作示意图;

[0028] 图5是UPSERT事件处理器CPU、GPU异步协同工作流程图；

[0029] 图6是DEL事件处理器CPU、GPU异步协同工作流程图。

具体实施方式

[0030] 为使本发明实施例的目的、技术方案和优点更加清楚,下面将结合本发明实施例中的附图,对本发明实施例中的技术方案进行清楚、完整地描述,显然,所描述的实施例是本发明一部分实施例,而不是全部的实施例,基于本发明中的实施例,本领域普通技术人员在没有做出创造性劳动的前提下所获得的所有其他实施例,都属于本发明保护的范围。

[0031] 本发明提出一种基于OpenCL实现时序数据降采样预计算的方法,如图1所示,负责异步处理REDISMODULE_EVENT_TS_SAMPLE类型事件的处理器EvtCompactionHandler; handleCompaction、upsertCompaction和compactionDelRange函数中CPU与GPU异步协同工作的方法;基于OpenCL标准和AggregationClass实现的CLAggSum、CLAggAvg、CLAggFirst等降采样计算器。本发明实现了一种异构处理器CPU、GPU的协同工作加速降采样时序数据预计算的方法。具体如下:

[0032] 1) 通过Redis Module的API增加了RedisModule_FireServerEvent函数来发送事件,定义了REDISMODULE_EVENT_TS_SAMPLE类型事件,包括如下三种事件:

[0033] • REDISMODULE_SUBEVENT_TS_SAMPLE_ADD

[0034] • REDISMODULE_SUBEVENT_TS_SAMPLE_UPSERT

[0035] • REDISMODULE_SUBEVENT_TS_SAMPLE_DEL

[0036] “TS.ADD/TS.MADD/TS.INCRE”、“TS.DEL”指令新增或删除时序数据点(SAMPLE),同步调用SeriesAddSample、SeriesUpsertSample、SeriesDelRange函数分别会触发ADD、UPSERET、DEL三种事件。在RedisModule_Onload函数中通过RedisModule_SubscribeToServer函数以回调函数的方式来注册上述事件的处理器EvtCompactionHandler,它根据上述不同的事件类型分别调用handleCompaction、upsertCompaction、compactionDelRange函数,下一步依据OpenCL启用标识来决定是否协调GPU异步执行相应的降采样聚合算法内核程序。

[0037] 2) 定义CPU、GPU异构处理器共享内存区域的内存管理器CLMemoryManager用来为CLAggSum、CLAggAvg、CLAggFirst、CLAggLast等降采样聚合算法内核的cl_context结构体申请和回收内存;内存管理器也管理CLSeries结构体中的Chunk结构体的内存空间,该空间为CPU与GPU的共享内存空间,避免在异构设备CPU及GPU之间发生数据拷贝。

[0038] 如附图2所示CompactionRule结构体完全由CPU来管理,CPU调用OpenCL标准的CLEnqueueMapBuffer函数将上述申请的共享内存空间映射为CompactionRule结构体中间接引用中的cl_context结构体的指针,以便GPU访问该共享内存空间。

[0039] 3) 如附图3中CompactionRule结构体中定义了cl_compaction标识定义该时序数据序列是否采用OpenCL标准AggregationClass结构体实现的CLAggSum、CLAggAvg、CLAggFirst等和降采样聚合算法内核。若需要降采样预计算的时间序列的量较多,GPU的算力无法满足时,通过动态调整cl_compaction标识降低GPU的负载,达到快速执行内核程序响应CPU协同的目的。

[0040] CLAggAvg降采样预计算方法为例:

- [0041] • createContext函数调用内存管理器为cl_context结构体申请内存空间,由CPU负责执行;
- [0042] • freeContext函数调用内存管理器释放前述申请的内存空间,由CPU负责执行;
- [0043] • appendValue内核函数将时序数据点追加到cl_context结构体中,由GPU负责执行;
- [0044] • finalize内核函数将cl_context中累积的时序数据计算出平均值,由GPU负责执行;
- [0045] • reset内核函数将cl_context中累积的时序数据清空、初始化,由GPU负责执行;
- [0046] • writeContext函数随着CompactionRule将CLAggAvg内核实例序列化至磁盘文件中,由CPU负责执行;
- [0047] • readContext函数从磁盘读取文件并反序列化CLAggAvg内核实例后将指针传递给CompactionRule,该函数由CPU负责执行;
- [0048] 4) 附图4、5、6分别为ADD、UPSERT、DEL事件处理器CPU、GPU异步协同工作流程图,图中CPU框内的函数由CPU负责执行,GPU框内的函数是基于OpenCL标准的内核函数。CPU与GPU除前述采用共享内存机制管理降采样算法上下文及时序数据块(chunk)外,GPU采用cl_command_queue内核执行队列异步接受协调指令调用执行内核函数。
- [0049] 以ADD事件处理器为例,处理流程描述如下:
- [0050] • CPU接收到源Key指向的时序数据Sample触发REDISMODULE_SUBEVENT_TS_SAMPLE_ADD事件;
- [0051] • EvtCompactionHandler处理器接收事件后,CPU负责执行handleCompaction处理函数;
- [0052] • handleCompaction函数将响应降采样聚合函数的appendValue内核程序以及Sample数据加入GPU的cl_command_queue队列;
- [0053] • GPU负责将Sample数据追加到降采样聚合函数的cl_context上下文中;
- [0054] • 循环执行前述追加Sample数据操作,直到到达合并规则定义的时间窗口时,CPU将响应降采样聚合函数的finalize内核程序加入GPU的cl_command_queue队列;
- [0055] • GPU负责执行finalize内核程序计算降采样聚合计算结果;
- [0056] • CPU通过执行clEnqueueUnmapMemObject函数获取前述降采样聚合计算结果;
- [0057] • CPU负责执行SeriesAddSample函数将降采样聚合计算结果写入目标key指向的时间序列中;
- [0058] 以上所述仅为本发明的较佳实施例,仅用于说明本发明的技术方案,并非用于限定本发明的保护范围。凡在本发明的精神和原则之内所做的任何修改、等同替换、改进等,均包含在本发明的保护范围内。

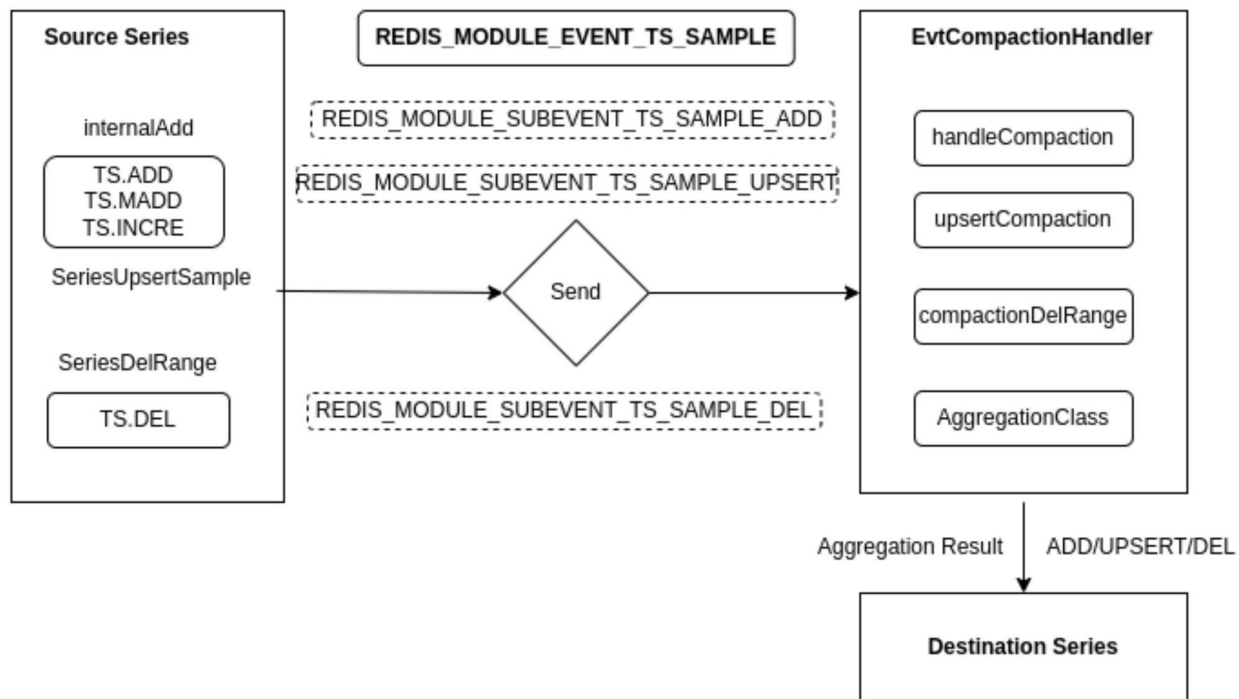


图1

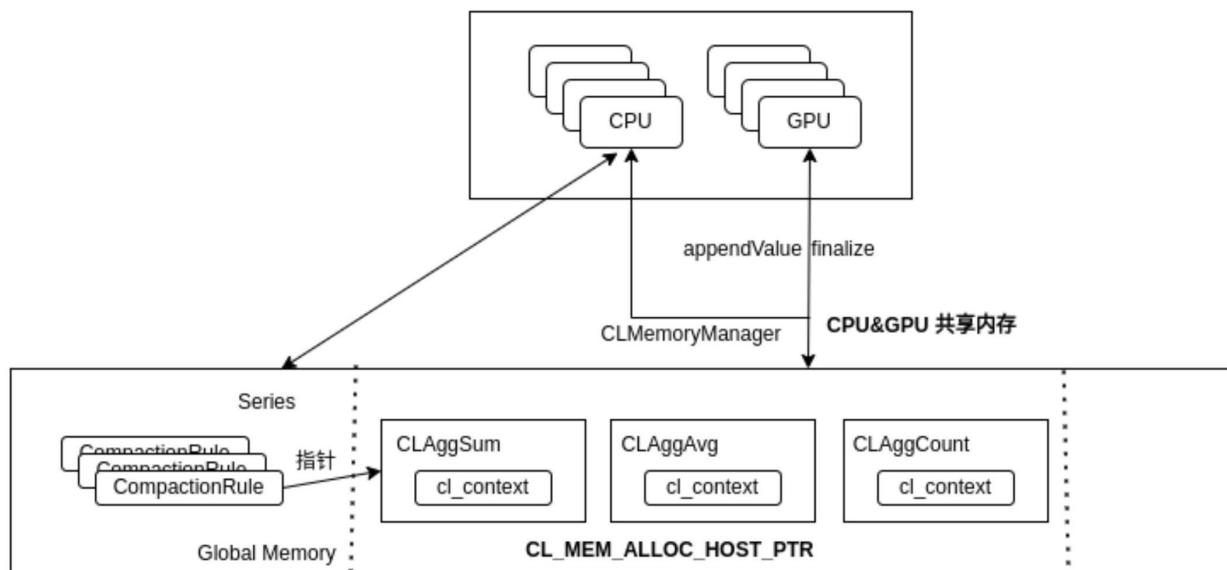


图2

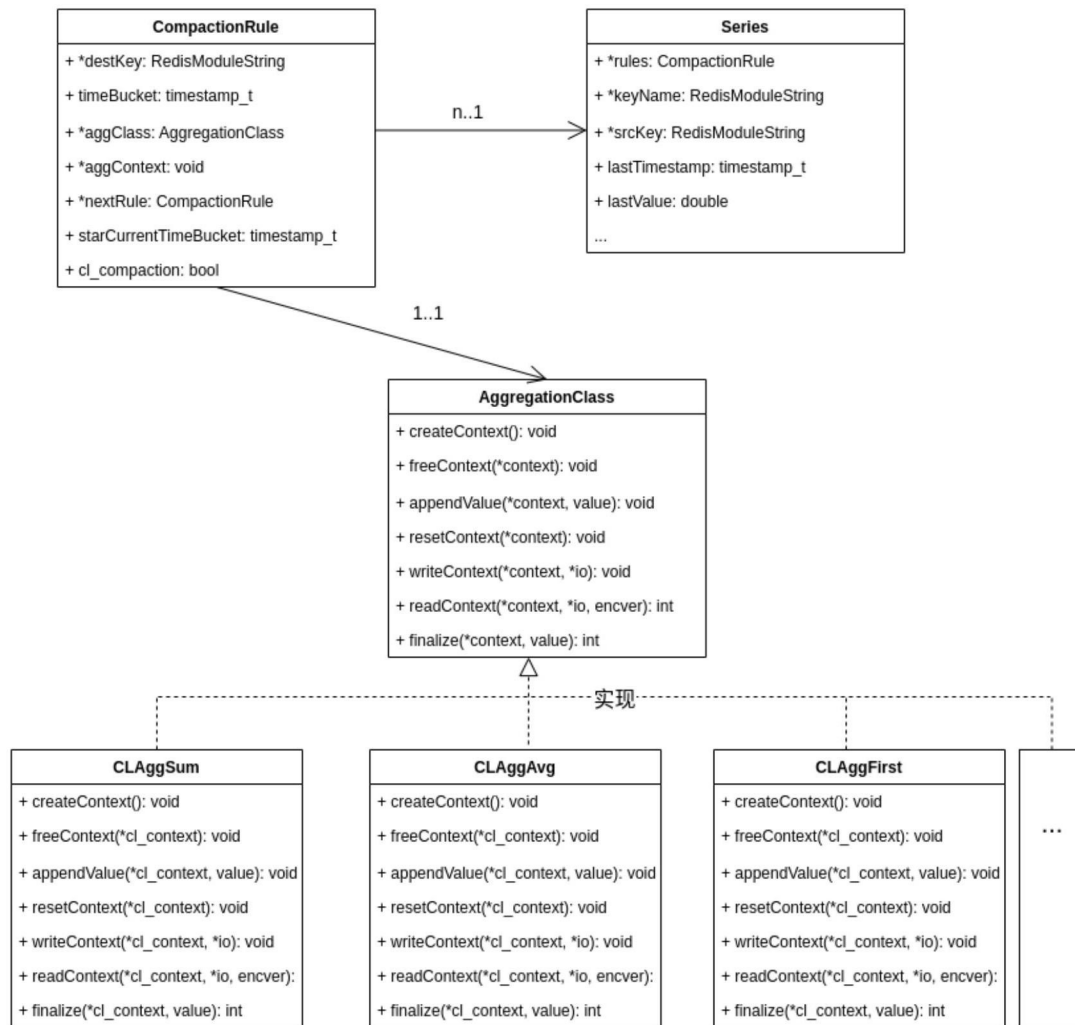


图3

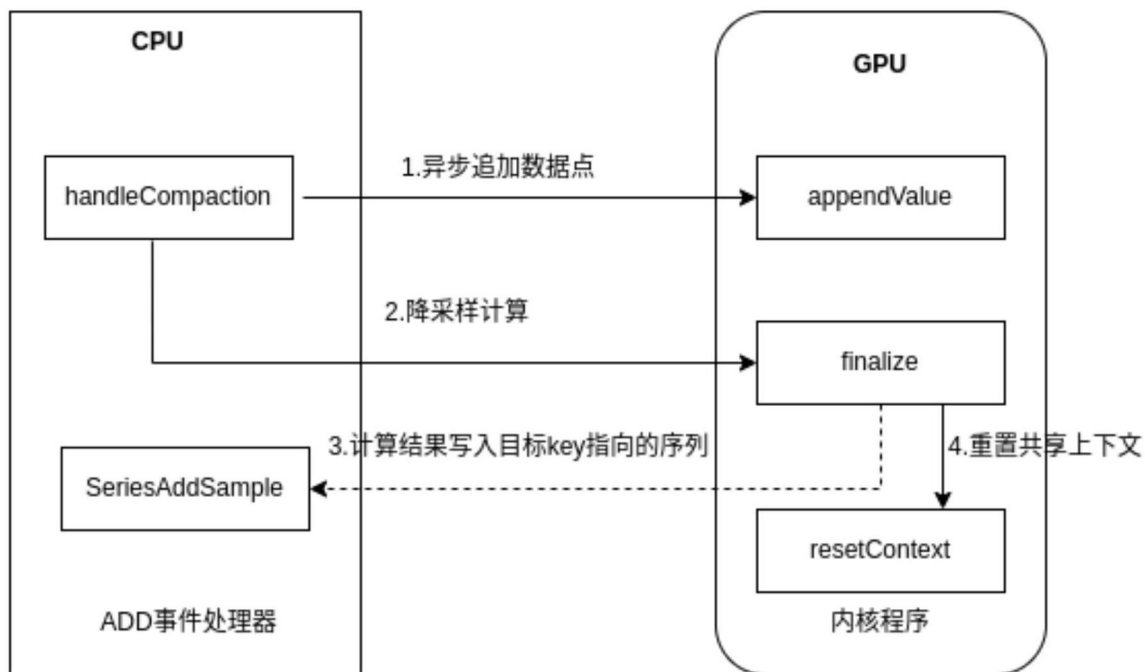


图4

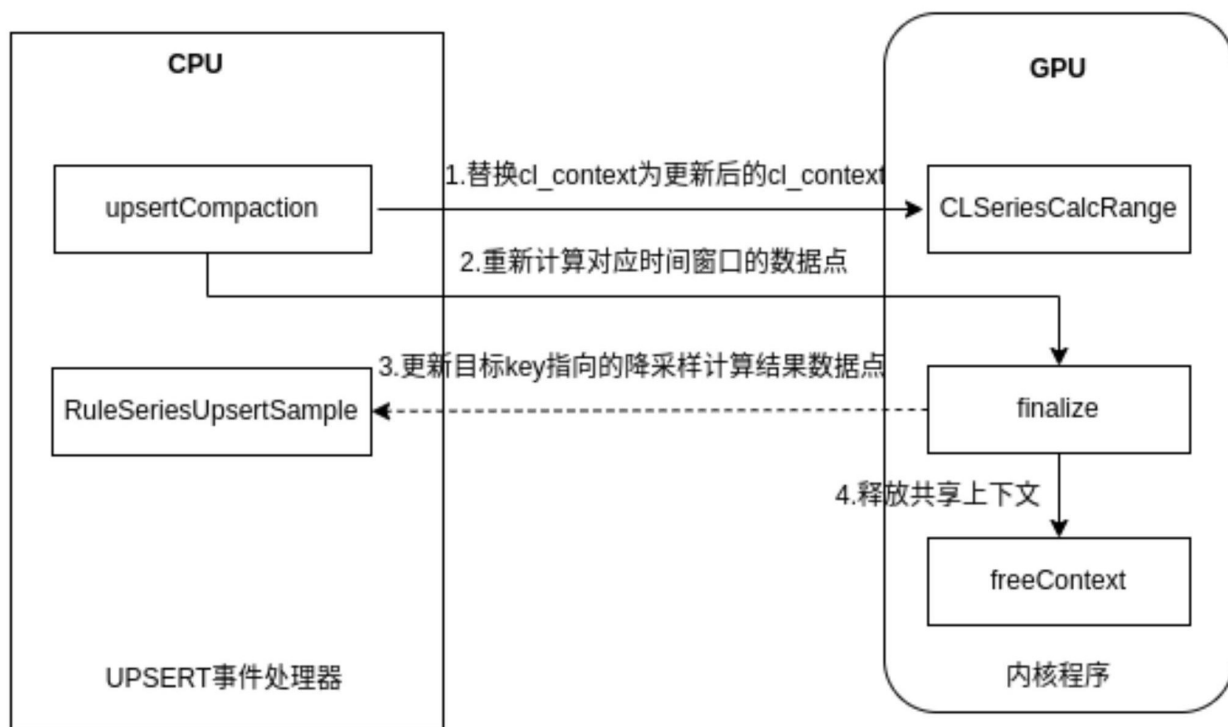


图5

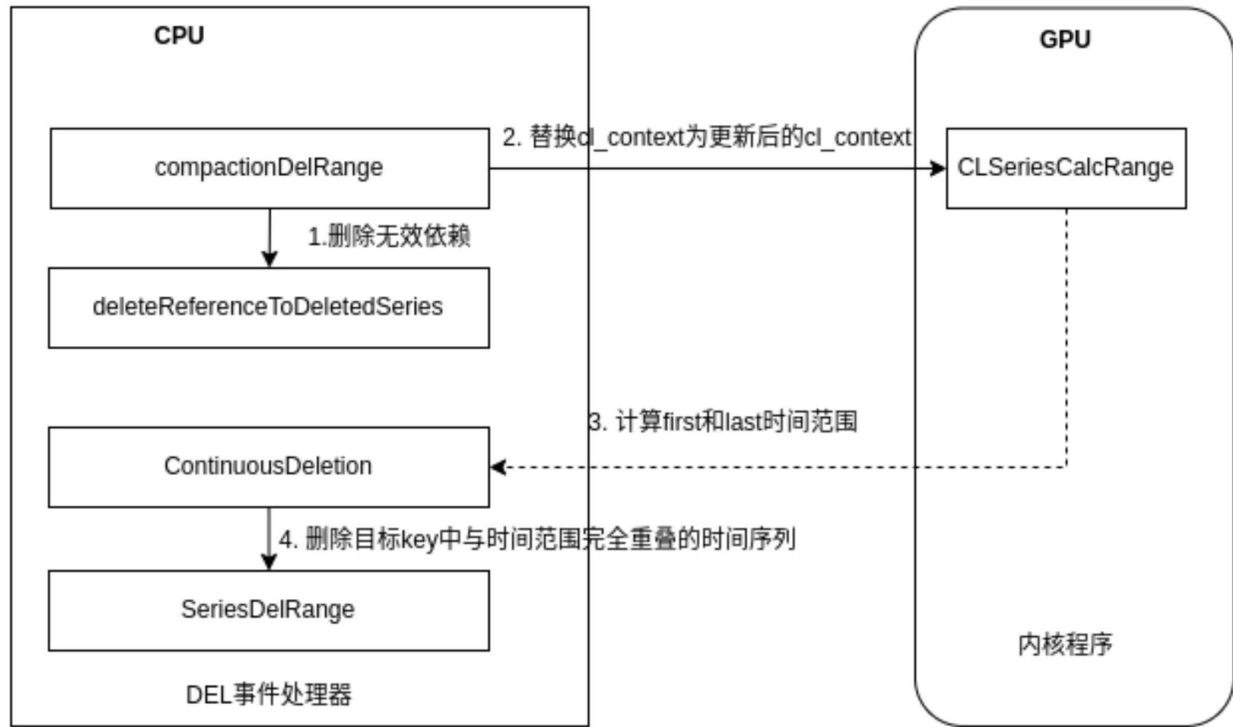


图6